



SK0-005

COMPTIA SERVER+

COURSE SLIDES · WSQ

CompTIA Certified Server+ Training

WSQ Course Code: TGS-2024048318

Conducted by Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Version v4.1 · 19 August 2026

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg



COURSE ADMINISTRATION

Welcome & Housekeeping

Digital Attendance (Mandatory)

- It is mandatory to take the AM, PM and Assessment digital attendance for WSQ-funded courses.
- The trainer/administrator displays the digital attendance QR code from the SSG portal.
- Scan the QR code with your mobile phone camera and submit your attendance.
- A minimum of 75% attendance is required to be eligible for assessment and funding.

About the Trainer



Your Trainer

General Trainer template —
to be completed by the trainer

NAME

TITLE / DESIGNATION

QUALIFICATIONS

AREAS OF EXPERTISE

TRAINING & INDUSTRY EXPERIENCE

CONTACT

About the Trainer



Dr Alfred Ang

Principal Trainer
Tertiary Infotech Academy Pte Ltd

ROLE

Principal Trainer, Tertiary Infotech Academy Pte Ltd

CERTIFICATION

CompTIA Server+ / infrastructure certified — server hardware, storage, virtualization and data-centre operations.

DELIVERS

WSQ courses on server administration, cloud infrastructure, networking and cybersecurity.

FOUNDER

Founder and lead instructor at Tertiary Infotech / Tertiary Courses.

Let's Know Each Other

- Your name and organisation / role.
- Your experience with servers, storage, networking or Linux/Windows administration (if any).
- What you want to be able to build, secure or troubleshoot after this course.

Ground Rules

1

Set your mobile phone to silent mode.

2

Participate actively — no question is too small.

3

Mutual respect: agree to disagree.

4

One conversation at a time.

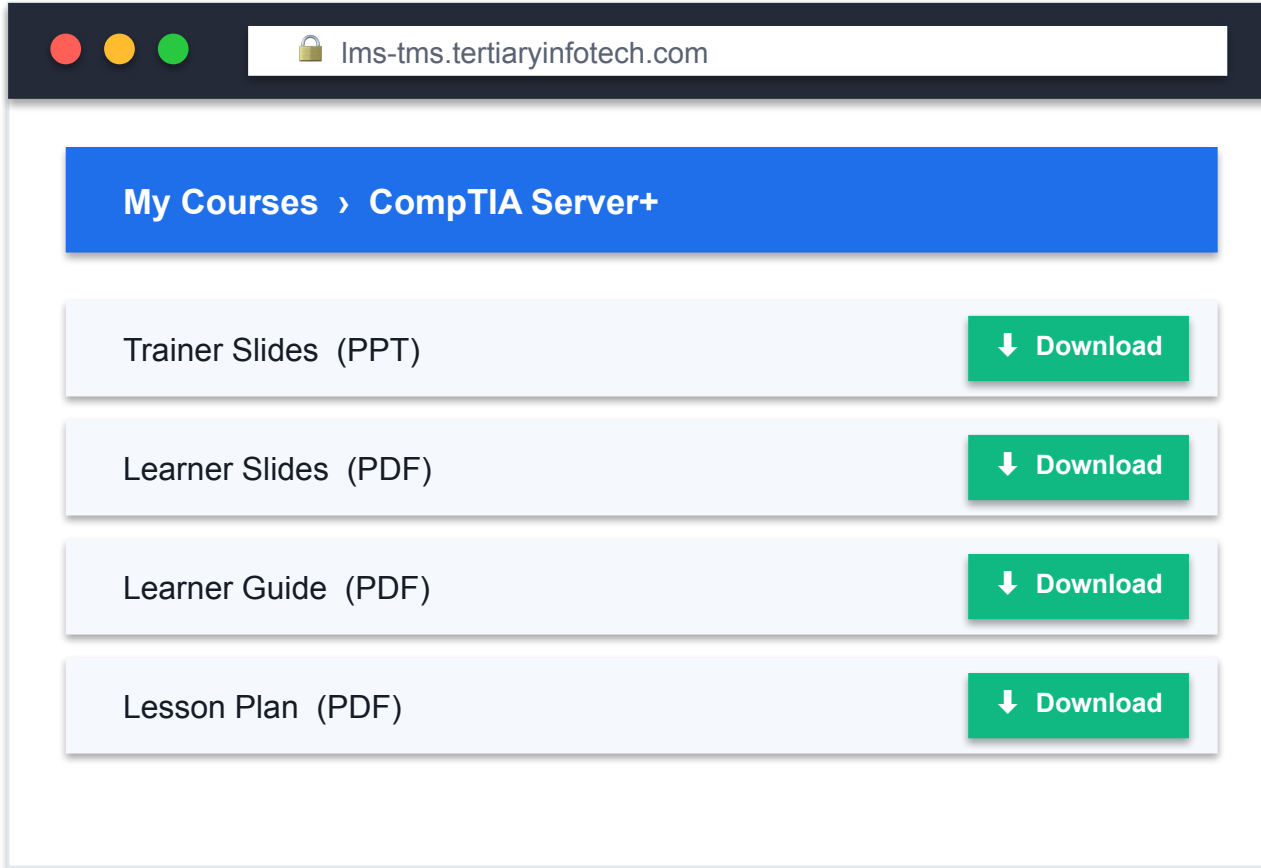
5

Be punctual; return from breaks on time.

6

75% attendance is required.

Download Your Course Material



The screenshot shows a web browser window with the address bar displaying `lms-tms.tertiaryinfotech.com`. The page content is organized into a blue header bar and a list of download items. The header bar contains the text "My Courses > CompTIA Server+". Below this, there are four items, each with a label and a green "Download" button with a downward arrow icon.

My Courses > CompTIA Server+	
Trainer Slides (PPT)	Download
Learner Slides (PDF)	Download
Learner Guide (PDF)	Download
Lesson Plan (PDF)	Download

1

Log in

Go to `lms-tms.tertiaryinfotech.com` and sign in with your enrolment email.

2

Open the course

Select CompTIA Certified Server+ Training from My Courses.

3

Download

Download the Trainer Slides, Learner Slides, Learner Guide and Lesson Plan.

4

Use in the open book

Keep the slides and Learner Guide open during the assessment.

SCHEDULE

Lesson Plan — 3 Days, 8 hours/day

Days 1–2

- Day 1 — Server Hardware, Storage & OS Administration
 - Topic 1: Server Hardware Installation & Management (Labs 1–5)
 - Topic 2: Server Administration begins (Labs 6–13)
- Day 2 — Networking, Virtualization, Security & Disaster Recovery
 - Topic 2: Server Administration (Labs 14–17)
 - Topic 3: Security & Disaster Recovery (Labs 18–27)

Day 3 & timing

- Day 3 — Troubleshooting & Final Assessment
 - Topic 4: Troubleshooting (Labs 28–33)
 - Revision and exam preparation
 - Final Assessment (WA + PP)
- Daily timing
 - 9:00am–6:00pm · 1-hour lunch · tea breaks within

Learning Outcomes

1

Hardware & Storage

Rack, cable, power and cool servers; deploy RAID and shared storage; manage firmware and out-of-band.

2

Server Administration

Install operating systems and file systems; configure IP/DNS/DHCP/VLANs; run roles, HA, virtualization and scripts.

3

Security & DR

Encrypt data, harden hosts, manage identity and access, decommission media, and plan backups and disaster recovery.

4

Troubleshooting

Diagnose hardware, storage, OS, software and network faults with a structured methodology.

5

Exam readiness

Map every hands-on skill to the CompTIA Server+ SK0-005 exam domains and pass the exam.

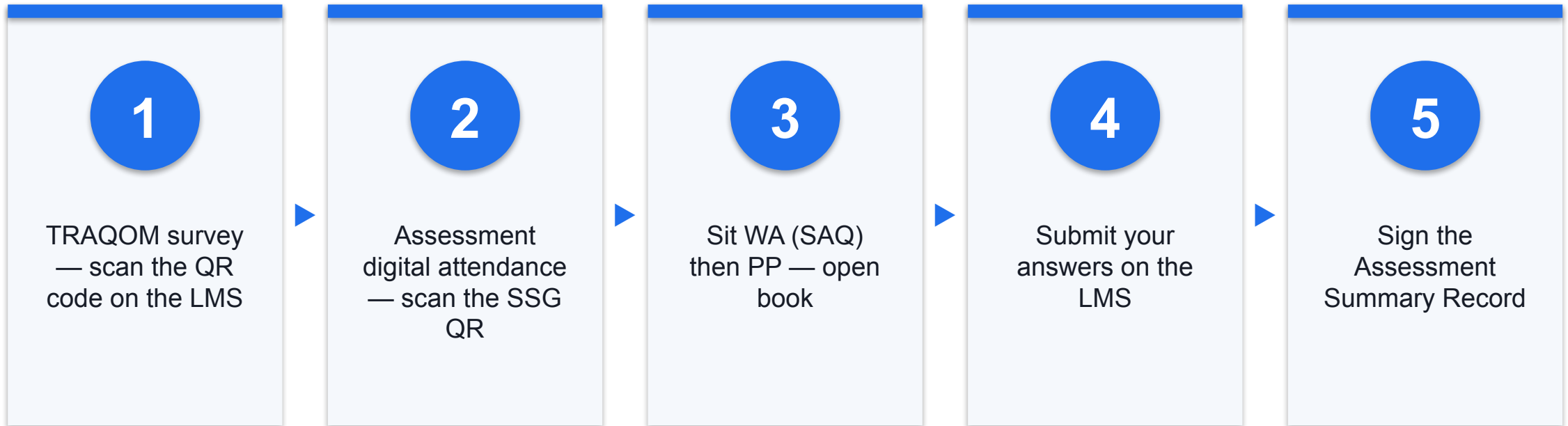
Briefing for Assessment

- Place phones and other materials under the table or on the floor.
- No photos or recording of assessment scripts.
- No discussion during the assessment.
- Use a black/blue pen for hard-copy assessments.
- No liquid paper / correction tape.
- Scripts are collected when time is up.

Assessment

- Written Assessment (WA) — Short-Answer Questions (SAQ), 1 hour, open book.
- Practical Performance (PP) — hands-on server administration tasks on the Killercoda Ubuntu playground, 1 hour, open book.
- Format: Open Book — slides, Learner Guide and approved materials only.
- A minimum of 75% attendance is required to be eligible for assessment and funding.
- An appeal process is available if required.

Assessment Flow





CORE CONCEPTS

Server Fundamentals

What is a Server?

1

Purpose-built

Hardware and an OS built to provide services — web, file, database, directory — reliably to many clients.

2

Availability first

Redundant power, storage and networking so the service stays up; measured against SLAs, RPO and RTO.

3

Managed remotely

Out-of-band controllers (IPMI/iDRAC/iLO) run the server even when the OS is down.

4

The SK0-005 role

Install, configure, secure and troubleshoot servers on-premises and in the cloud.

The Four Server+ Exam Domains

Install & administer

- Hardware, racking, power
- Storage & RAID, OS install
- Networking, roles, HA, scripting

Secure & recover

- Data & physical security
- IAM, hardening, decommissioning
- Backup & disaster recovery

Troubleshoot

- Structured 6-step method
- Hardware, storage, OS faults
- Network faults & DR validation

WHY IT MATTERS

Install, administer, secure, troubleshoot.

The four SK0-005 domains follow the real life of a server — from racking and OS install to hardening, backup and fault diagnosis.

On-premises vs. cloud

Your data centre

- On-premises
 - You own the hardware and data centre
 - Full control of security and latency
 - Capital cost, capacity planning, maintenance

Public / private / hybrid

- Cloud (IaaS/PaaS/SaaS)
 - Rented infrastructure, pay-as-you-go
 - Elastic scaling and managed services
 - Shared-responsibility security model

Your Lab Environment — Killercoda

- Every hands-on lab runs on the free Killercoda Ubuntu playground — a disposable Linux VM in the browser.
- Portal: <https://killercoda.com/playgrounds/scenario/ubuntu> — no local install or physical hardware needed.
- Install any tool with apt inside the throw-away VM; reset the playground between state-changing labs.
- Hardware-only concepts use software equivalents: loopback disks for RAID, ip link for NICs, KVM for VMs, dmidecode/ipmitool for hardware inspection.

Lab Environments

1

Linux — Killercode Ubuntu

All 33 core labs. A disposable Ubuntu VM in the browser: killercode.com/playgrounds/scenario/ubuntu

2

Kubernetes — Killercode

Bonus container-orchestration assets (Labs 14 & 15): killercode.com/playgrounds/scenario/kubernetes

3

Docker — Docker Desktop

Bonus Dockerfiles & Compose (Labs 10 & 15). Install: docker.com/products/docker-desktop

4

What ships in each lab

Its own folder: guide (.md) + guidelines/cheatsheet/worksheet PDFs, CSV+Excel data, and scripts / YAML / Dockerfiles where relevant.

Free Tools You'll Use

1

Hardware & storage

dmidecode, lshw, ipmitool, mdadm, lvm2, parted, smartctl — inspect, build and monitor server hardware and disks.

2

Networking

ip, bind9, isc-dhcp-server, ufw/nftables, tcpdump, dig, mtr — configure and diagnose server networking.

3

Roles & virtualization

nginx, samba, mariadb, slapd, qemu-kvm/libvirt, haproxy/keepalived — stand up services and high availability.

4

Security

cryptsetup, openssl, sudo/PAM, auditd, lynis, unattended-upgrades, shred/nwipe — encrypt, harden and sanitize.

5

Backup & DR

tar, rsync, restic, lsyncd — full/incremental backups, replication and site failover.

6

Online helpers

IP Calculator (alfredang.github.io/ipcalculator), the Cybersecurity Simulator, and the Pcap/Regex tools.

TOPIC 01

Server Hardware Installation and Management

Racking & power · storage & RAID · out-of-band management & firmware

Key Concepts — Server Hardware Installation and Management

1

Form factors & racking

Rack (1U/2U/4U) vs. tower vs. blade; rack units, rail kits, cable management, weight distribution and airflow direction (front-to-back).

2

Power & cooling

Redundant PSUs, dual power feeds, PDUs and UPS; hot vs. cold aisle containment; environmental targets for temperature and humidity.

3

RAID levels

RAID 0/1/5/6/10 and JBOD — capacity and fault-tolerance formulas; hardware vs. software RAID; when RAID 6 or RAID 10 wins.

4

Shared storage

DAS vs. NAS vs. SAN; NFS/SMB (file) vs. iSCSI/Fibre Channel (block); LUNs, and capacity planning with growth headroom.

5

Out-of-band management

IPMI/BMC, iDRAC/iLO for lights-out control; firmware/BIOS/UEFI updates and the update order.

6

Hot-swap & maintenance

Hot-swappable drives, PSUs and fans; hot-add vs. cold maintenance; component replacement without downtime.

Hands-On Labs — Server Hardware Installation and Management

Labs 1–2

- Server Form Factors, Racking, and Power Planning
- Storage Deployment: RAID 0/1/5/6/10 with mdadm

Labs 3–4

- LVM Provisioning and Capacity Planning
- Shared Storage: NAS (NFS/CIFS) and SAN (iSCSI)

Labs 5–5

- Out-of-Band Management, BIOS/UEFI, and Firmware

Server Form Factors, Racking, and Power Planning

LAB 1

The learner inventories a running system with dmidecode/lshw and plans a 42U rack layout with power and cooling rules the way a data-centre technician would.

You'll build

A rack layout plan plus a hardware inventory and power-budget sanity check.

Tools: dmidecode, lshw, lscpu, lspci, lsusb

Server Form Factors, Racking, and Power Planning

STEP 1 OF 8

1

Install the hardware inventory tools

```
$ apt update && apt install -y dmidecode lshw pciutils usbutils
```

Server Form Factors, Racking, and Power Planning

STEP 2 OF 8

2

Identify the running system as a rack-mount server

```
$ dmidecode -t system | grep -E "Manufacturer|Product|Serial"
```

Server Form Factors, Racking, and Power Planning

STEP 3 OF 8

3

Read chassis type and U-height

```
$ dmidecode -t chassis | grep -E "Type|Height|Power"
```

Server Form Factors, Racking, and Power Planning

STEP 4 OF 8

4

Inventory CPU, RAM, bus types and expansion cards

```
$ lscpu | head -20
```

Server Form Factors, Racking, and Power Planning

STEP 5 OF 8

5

List PCI/PCIe buses and cards

```
$ lspci | head
```

Server Form Factors, Racking, and Power Planning

STEP 6 OF 8

6

Get the model string for a hardware compatibility list (HCL) check

```
$ dmidecode -s system-product-name
```

Server Form Factors, Racking, and Power Planning

STEP 7 OF 8



Plan a 42U rack layout: heaviest at bottom, redundant power on separate circuits

Server Form Factors, Racking, and Power Planning

STEP 8 OF 8



8

Do a power-budget sanity check against the PDU and breaker (80% rule)

Server Form Factors, Racking, and Power Planning

Test it

The learner verifies success by reading chassis/CPU/memory data and producing a rack layout that respects U sizing, weight balancing, cooling and redundant power.

Storage Deployment: RAID 0/1/5/6/10 with mdadm

LAB 2

The learner builds every RAID level the exam tests on loopback files with mdadm, then fails and rebuilds an array to observe resync.

You'll build

Working RAID 0/1/5/6/10 and JBOD arrays plus a completed rebuild.

Tools: mdadm, losetup, mkfs.ext4

Storage Deployment: RAID 0/1/5/6/10 with mdadm

STEP 1 OF 9

1

Install mdadm

```
$ apt update && apt install -y mdadm
```

Storage Deployment: RAID 0/1/5/6/10 with mdadm

STEP 2 OF 9

2

Create six loopback disks that behave like drives

```
$ for i in 1 2 3 4 5 6; do truncate -s 200M disk$i.img; losetup /dev/loop$i disk$i.img; done
```

Storage Deployment: RAID 0/1/5/6/10 with mdadm

STEP 3 OF 9

3

Build a RAID 0 stripe (N x disk, no redundancy)

```
$ mdadm --create /dev/md0 --level=0 --raid-devices=2 /dev/loop1 /dev/loop2
```

Storage Deployment: RAID 0/1/5/6/10 with mdadm

STEP 4 OF 9

4

Build a RAID 1 mirror (survives 1 disk failure)

```
$ mdadm --create /dev/md1 --level=1 --raid-devices=2 /dev/loop1 /dev/loop2
```

Storage Deployment: RAID 0/1/5/6/10 with mdadm

STEP 5 OF 9

5

Build a RAID 5 single-parity array

```
$ mdadm --create /dev/md5 --level=5 --raid-devices=3 /dev/loop1 /dev/loop2 /dev/loop3
```


Storage Deployment: RAID 0/1/5/6/10 with mdadm

STEP 6 OF 9

6

Build a RAID 6 double-parity array

```
$ mdadm --create /dev/md6 --level=6 --raid-devices=4 /dev/loop1 /dev/loop2 /dev/loop3 /dev/loop4
```

Storage Deployment: RAID 0/1/5/6/10 with mdadm

STEP 7 OF 9

7

Build a RAID 10 stripe of mirrors

```
$ mdadm --create /dev/md10 --level=10 --raid-devices=4 /dev/loop1 /dev/loop2 /dev/loop3 /dev/loop4
```

Storage Deployment: RAID 0/1/5/6/10 with mdadm

STEP 8 OF 9

8

Simulate a failure and rebuild, watching resync

```
$ mdadm /dev/md1 --fail /dev/loop2 --remove /dev/loop2
```

Storage Deployment: RAID 0/1/5/6/10 with mdadm

STEP 9 OF 9

A green circle with a white number 9 inside, indicating the current step in the lab.

Compare hardware RAID vs software RAID trade-offs

Storage Deployment: RAID 0/1/5/6/10 with mdadm

Test it

The learner verifies success by reading the array status in `/proc/mdstat` and confirming the capacity and fault-tolerance formula for each level.

LVM Provisioning and Capacity Planning

LAB 3

The learner provisions storage through the PV to VG to LV model, extends a volume online, adds swap and takes a point-in-time snapshot.

You'll build

An LVM volume group with web, database and swap volumes plus a live snapshot.

Tools: lvm2, mkfs.ext4, mkfs.xfs

LVM Provisioning and Capacity Planning

STEP 1 OF 9

1

Install lvm2

```
$ apt update && apt install -y lvm2
```

LVM Provisioning and Capacity Planning

STEP 2 OF 9

2

Create four loopback disks

```
$ for i in 1 2 3 4; do truncate -s 500M disk$i.img; losetup /dev/loop$i disk$i.img; done
```


LVM Provisioning and Capacity Planning

STEP 3 OF 9

3

Create physical volumes and a volume group

```
$ vgcreate vg_data /dev/loop1 /dev/loop2 /dev/loop3
```

LVM Provisioning and Capacity Planning

STEP 4 OF 9

4

Create logical volumes for web and database

```
$ lvcreate -L 800M -n lv_web vg_data
```

LVM Provisioning and Capacity Planning

STEP 5 OF 9

5

Extend the DB volume online (add PV, grow LV, grow FS)

```
$ lvextend -L +400M /dev/vg_data/lv_db
```

LVM Provisioning and Capacity Planning

STEP 6 OF 9

6

Grow the XFS filesystem online

```
$ xfs_growfs /srv/db
```

LVM Provisioning and Capacity Planning

STEP 7 OF 9

7

Create a dedicated swap LV

```
$ lvcreate -L 200M -n lv_swap vg_data
```

LVM Provisioning and Capacity Planning

STEP 8 OF 9

8

Take a point-in-time snapshot of a live volume

```
$ lvcreate -L 100M -s -n lv_web_snap /dev/vg_data/lv_web
```

LVM Provisioning and Capacity Planning

STEP 9 OF 9

9

Report capacity vs utilisation as a baseline

```
$ vgs --units g -o vg_name,vg_size,vg_free
```

LVM Provisioning and Capacity Planning

Test it

The learner verifies success by confirming the extended volume grew online and the snapshot restores a deleted file.

Shared Storage: NAS (NFS/CIFS) and SAN (iSCSI)

LAB 4

The learner exports file-level NFS and CIFS shares and builds a block-level iSCSI target, then connects an initiator and formats the LUN as a disk.

You'll build

A working NFS share, a Samba/CIFS share and a mounted iSCSI SAN LUN.

Tools: nfs-kernel-server, samba, tgt, open-iscsi

Shared Storage: NAS (NFS/CIFS) and SAN (iSCSI)

STEP 1 OF 9

1

Install the NAS and SAN packages

```
$ apt update && apt install -y nfs-kernel-server nfs-common tgt open-iscsi samba
```

Shared Storage: NAS (NFS/CIFS) and SAN (iSCSI)

STEP 2 OF 9

2

Export an NFS share (file-level NAS)

```
$ echo "/srv/nfs/share 127.0.0.1(rw,sync,no_subtree_check,no_root_squash)" >> /etc/exports
```

Shared Storage: NAS (NFS/CIFS) and SAN (iSCSI)

STEP 3 OF 9

3

Mount the NFS share as a client

```
$ mount -t nfs 127.0.0.1:/srv/nfs/share /mnt/nas
```

Shared Storage: NAS (NFS/CIFS) and SAN (iSCSI)

STEP 4 OF 9

4

Create a CIFS/SMB share and browse it

```
$ smbclient -L //127.0.0.1 -N
```

Shared Storage: NAS (NFS/CIFS) and SAN (iSCSI)

STEP 5 OF 9

5

Build an iSCSI target with a 500 MB LUN

```
$ systemctl restart tgt
```

Shared Storage: NAS (NFS/CIFS) and SAN (iSCSI)

STEP 6 OF 9

6

Discover and log in to the iSCSI target

```
$ iscsiadm -m discovery -t st -p 127.0.0.1
```

Shared Storage: NAS (NFS/CIFS) and SAN (iSCSI)

STEP 7 OF 9

7

Log in to the iSCSI node

```
$ iscsiadm -m node --login
```


Shared Storage: NAS (NFS/CIFS) and SAN (iSCSI)

STEP 8 OF 9

8

Format and mount the new SAN LUN as a local disk

```
$ mkfs.ext4 /dev/$NEW
```

Shared Storage: NAS (NFS/CIFS) and SAN (iSCSI)

STEP 9 OF 9

9

Compare NAS vs SAN vs Fibre Channel/FCoE

Shared Storage: NAS (NFS/CIFS) and SAN (iSCSI)

Test it

The learner verifies success by reading files over the NFS/CIFS mounts and confirming a new /dev/sdX LUN appears from the iSCSI target and mounts.

Out-of-Band Management, BIOS/UEFI, and Firmware

LAB 5

The learner inspects BIOS/UEFI from the OS, practises the efibootmgr and ipmitool workflows, and walks the fwupd firmware-upgrade and hot-swap checklists.

You'll build

A firmware/BIOS inventory and a documented OOB management and hot-swap workflow.

Tools: ipmitool, fwupd, efibootmgr, dmidecode

Out-of-Band Management, BIOS/UEFI, and Firmware

STEP 1 OF 9

1

Install the OOB and firmware tools

```
$ apt update && apt install -y ipmitool fwupd efibootmgr dmidecode
```

Out-of-Band Management, BIOS/UEFI, and Firmware

STEP 2 OF 9

2

Inventory BIOS/UEFI from the running OS

```
$ dmidecode -t bios
```

Out-of-Band Management, BIOS/UEFI, and Firmware

STEP 3 OF 9

3

Confirm UEFI vs legacy BIOS boot mode

```
$ [ -d /sys/firmware/efi ] && echo "Booted in UEFI mode" || echo "Booted in legacy BIOS mode"
```

Out-of-Band Management, BIOS/UEFI, and Firmware

STEP 4 OF 9

4

List UEFI boot entries

```
$ efibootmgr -v
```


Out-of-Band Management, BIOS/UEFI, and Firmware

STEP 5 OF 9

5

Practise the IPMI/BMC chassis-status pattern

```
$ ipmitool -H bmc.example.com -U admin -P 'CHANGEME' chassis status
```

Out-of-Band Management, BIOS/UEFI, and Firmware

STEP 6 OF 9

6

List which firmware is installed with fwupd

```
$ fwupdmgr get-devices
```

Out-of-Band Management, BIOS/UEFI, and Firmware

STEP 7 OF 9

7

Check for available firmware updates

```
$ fwupdmgr get-updates
```

Out-of-Band Management, BIOS/UEFI, and Firmware

STEP 8 OF 9



8

Walk the four local-admin paths (crash cart, KVM, serial, virtual console)

Out-of-Band Management, BIOS/UEFI, and Firmware

STEP 9 OF 9

9

Walk the hot-swap pre-swap checklist for drives, PSUs and fans

Out-of-Band Management, BIOS/UEFI, and Firmware

Test it

The learner verifies success by reading the BIOS version and boot mode and mapping each ipmitool command to its Server+ OOB sub-objective.

Recap — Server Hardware Installation and Management

- You can now: 1.1 Install physical hardware
- You can now: 1.2 Deploy and manage storage (RAID levels, JBOD, capacity planning)
- You can now: 1.2/2.1/2.3 Capacity planning and logical volume management
- You can now: 1.2 Shared storage: NAS (NFS, CIFS), SAN (iSCSI)
- You can now: 1.3 Perform server hardware maintenance (OOB, BIOS/UEFI, firmware, hot-swap)

TOPIC 02

Server Administration

OS install & file systems · networking · roles, HA, virtualization, scripting & assets

Key Concepts — Server Administration

1

OS installation

Attended, unattended, scripted and image-based installs; partition schemes (MBR vs. GPT); file systems (ext4, XFS, ZFS, NTFS).

2

Network services

IPv4/IPv6 addressing and subnetting, DNS, DHCP, FQDN, VLANs and default gateways for a server.

3

Server roles & monitoring

Web, file, database, directory and print roles; performance baselining, metrics and thresholds; data migration.

4

High availability

Clustering, load balancing, NIC teaming and failover to remove single points of failure.

5

Virtualization & cloud

Host vs. guest, hypervisor types, resource allocation and over-commit; IaaS/PaaS/SaaS and public/private/hybrid models.

6

Scripting & asset management

Variables, loops and conditionals for common server tasks; documentation, lifecycle and secure asset storage.

Hands-On Labs — Server Administration

Labs 6–9

- Installing a Server OS (Unattended / Scripted)
- Partition and Volume Types, File Systems
- IP, VLAN, DNS, DHCP, FQDN, and Hosts File
- Server Firewall and Port Management

Labs 10–13

- Server Roles: Web, File, and Database
- Directory Services with OpenLDAP
- Performance Monitoring, Baselineing, Event Logs
- Data Migration and Transfer

Labs 14–17

- High Availability: Clustering, Load Balancing, NIC Teaming
- Virtualization with KVM / QEMU
- Scripting Basics for Server Administration
- Asset Management, Documentation, and Licensing

Installing a Server OS (Unattended / Scripted)

LAB 6

The learner validates minimum specs and the HCL, builds a cloud-init autoinstall seed.iso, captures a golden image and walks the P2V conversion workflow.

You'll build

A cloud-init autoinstall seed.iso and a golden-image tarball for template deployment.

Tools: debootstrap, cloud-init, genisoimage, qemu-utils

Installing a Server OS (Unattended / Scripted)

STEP 1 OF 9

1

Install the OS-deployment tools

```
$ apt update && apt install -y debootstrap cloud-init genisoimage qemu-utils
```

Installing a Server OS (Unattended / Scripted)

STEP 2 OF 9

2

Confirm minimum OS requirements (CPU, RAM, disk)

```
$ lscpu | grep -E "Architecture|CPU(s)"
```

Installing a Server OS (Unattended / Scripted)

STEP 3 OF 9

3

Do an HCL check on CPU, NIC and storage controller

```
$ echo "NIC: $(lspci | grep -i ether)"
```

Installing a Server OS (Unattended / Scripted)

STEP 4 OF 9

4

Build a minimal cloud-init autoinstall config

Installing a Server OS (Unattended / Scripted)

STEP 5 OF 9

5

Package the config into a NoCloud seed ISO

```
$ genisoimage -output seed.iso -volid cidata -joliet -rock user-data meta-data
```


Installing a Server OS (Unattended / Scripted)

STEP 6 OF 9

6

Capture a golden image of the root filesystem

```
$ tar --one-file-system --exclude=/proc --exclude=/sys --exclude=/dev --exclude=/mnt -czf /srv/golden.tgz / 2>/dev/null
```

Installing a Server OS (Unattended / Scripted)

STEP 7 OF 9

7

Build a template root filesystem with debootstrap

```
$ debootstrap --variant=minbase jammy /srv/template http://archive.ubuntu.com/ubuntu/
```

Installing a Server OS (Unattended / Scripted)

STEP 8 OF 9

8

Create a destination image shape for P2V conversion

```
$ qemu-img create -f qcow2 /srv/converted.qcow2 5G
```

Installing a Server OS (Unattended / Scripted)

STEP 9 OF 9

9

Compare GUI vs Core vs Bare metal vs Virtualized vs Remote installs

Installing a Server OS (Unattended / Scripted)

Test it

The learner verifies success by confirming the seed.iso is built and the golden-image tarball and debootstrap template are deployable filesystems.

Partition and Volume Types, File Systems

LAB 7

The learner partitions a disk with MBR then GPT, formats ext4/XFS/ZFS, persists mounts by UUID and converts MBR to GPT non-destructively.

You'll build

A GPT-partitioned disk with ext4, XFS and ZFS filesystems and a UUID-based fstab entry.

Tools: parted, gdisk, e2fsprogs, xfsprogs, zfsutils-linux

Partition and Volume Types, File Systems

STEP 1 OF 9

1

Install the partitioning and filesystem tools

```
$ apt update && apt install -y parted gdisk e2fsprogs xfsprogs zfsutils-linux ntfs-3g
```

Partition and Volume Types, File Systems

STEP 2 OF 9

2

Create a 4G loopback disk to partition

```
$ losetup -P /dev/loop10 /srv/disk.img
```


Partition and Volume Types, File Systems

STEP 3 OF 9

3

Create an MBR (msdos) partition table

```
$ parted -s /dev/loop10 mklabel msdos
```

Partition and Volume Types, File Systems

STEP 4 OF 9

4

Switch the disk to a GPT partition table

```
$ parted -s /dev/loop10 mklabel gpt
```

Partition and Volume Types, File Systems

STEP 5 OF 9

5

Format the ext4 partition

```
$ mkfs.ext4 /dev/loop10p1
```

Partition and Volume Types, File Systems

STEP 6 OF 9

6

Create a ZFS pool on a partition

```
$ zpool create -f tank /dev/loop10p3
```

Partition and Volume Types, File Systems

STEP 7 OF 9



Compare the five Server+ filesystems (ext4, NTFS, VMFS, ReFS, ZFS)

Partition and Volume Types, File Systems

STEP 8 OF 9

8

Persist a mount by UUID in /etc/fstab

```
$ echo "UUID=$UUID /mnt/p1 ext4 defaults,noatime 0 2" | tee -a /etc/fstab
```

Partition and Volume Types, File Systems

STEP 9 OF 9

9

Convert MBR to GPT non-destructively with gdisk

```
$ gdisk -l /dev/loop10 | head
```

Partition and Volume Types, File Systems

Test it

The learner verifies success by printing the partition table with parted/gdisk and confirming the UUID-based fstab mount survives mount -a.

IP, VLAN, DNS, DHCP, FQDN, and Hosts File

LAB 8

The learner configures a static IP and gateway, tags a VLAN, builds a BIND9 zone with A records and a DHCP scope, and simulates APIPA.

You'll build

A configured static IP with VLAN, a BIND9 zone serving an FQDN and a validated DHCP scope.

Tools: iproute2, bind9, dnsutils, isc-dhcp-server

IP, VLAN, DNS, DHCP, FQDN, and Hosts File

STEP 1 OF 9

1

Install the networking service packages

```
$ apt update && apt install -y iproute2 vlan bind9 bind9utils dnsutils isc-dhcp-server
```

IP, VLAN, DNS, DHCP, FQDN, and Hosts File

STEP 2 OF 9

2

Inspect IP configuration, routes and MAC addresses

```
$ ip -c addr
```

IP, VLAN, DNS, DHCP, FQDN, and Hosts File

STEP 3 OF 9

3

Configure a static IP and default gateway

```
$ ip addr add 10.99.99.10/24 dev eth0
```

IP, VLAN, DNS, DHCP, FQDN, and Hosts File

STEP 4 OF 9

4

Tag VLAN 20 on a single NIC with 802.1Q

```
$ ip link add link eth0 name eth0.20 type vlan id 20
```

IP, VLAN, DNS, DHCP, FQDN, and Hosts File

STEP 5 OF 9

5

Review RFC 1918 private ranges and APIPA

IP, VLAN, DNS, DHCP, FQDN, and Hosts File

STEP 6 OF 9

6

Build a BIND9 zone with A records and restart

```
$ systemctl restart bind9
```

IP, VLAN, DNS, DHCP, FQDN, and Hosts File

STEP 7 OF 9

7

Resolve the FQDN against the local server

```
$ dig @127.0.0.1 web.lab.local +short
```


IP, VLAN, DNS, DHCP, FQDN, and Hosts File

STEP 8 OF 9

8

Add a hosts-file override that beats DNS

```
$ echo "10.99.99.50 app.lab.local app" >> /etc/hosts
```

IP, VLAN, DNS, DHCP, FQDN, and Hosts File

STEP 9 OF 9

9

Validate an ISC DHCP scope config

```
$ dhcpd -t -cf /etc/dhcp/dhcpd.conf && echo "Config OK"
```

IP, VLAN, DNS, DHCP, FQDN, and Hosts File

Test it

The learner verifies success by resolving web.lab.local through BIND9 and confirming the DHCP config passes validation.

Server Firewall and Port Management

LAB 9

The learner memorises the well-known ports, applies a default-deny UFW policy, translates it to raw nftables and audits listening vs reachable vs permitted.

You'll build

A default-deny firewall permitting only SSH/HTTP/HTTPS with source-restricted rules.

Tools: ufw, nftables, nginx, nmap

Server Firewall and Port Management

STEP 1 OF 9

1

Install the firewall and scan tools

```
$ apt update && apt install -y ufw nftables nginx nmap
```

Server Firewall and Port Management

STEP 2 OF 9



2

Review the well-known server port table

Server Firewall and Port Management

STEP 3 OF 9

3

Start a service and observe its open port

```
$ ss -tlnp | grep nginx
```

Server Firewall and Port Management

STEP 4 OF 9

4

Apply a default-deny UFW policy with explicit allows

```
$ ufw default deny incoming
```


Server Firewall and Port Management

STEP 5 OF 9

5

Allow SSH, HTTP and HTTPS

```
$ ufw allow 22/tcp comment 'SSH admin'
```

Server Firewall and Port Management

STEP 6 OF 9

6

Confirm the policy with nmap

```
$ nmap -p 22,23,25,80,443,3306 127.0.0.1
```

Server Firewall and Port Management

STEP 7 OF 9

7

Write the equivalent rule in raw nftables

```
$ nft add rule inet filter input tcp dport 8080 accept
```

Server Firewall and Port Management

STEP 8 OF 9

8

Add a source-restricted rule for MySQL

```
$ ufw allow from 10.99.99.0/24 to any port 3306 proto tcp comment 'MySQL app tier'
```

Server Firewall and Port Management

STEP 9 OF 9

9

Run the 3-view audit: listening, reachable, permitted

```
$ ss -tulnp
```

Server Firewall and Port Management

Test it

The learner verifies success by confirming nmap sees only the allowed ports open and the ss, nmap and ufw views agree.

Server Roles: Web, File, and Database

LAB 10

The learner stands up nginx, Samba and MariaDB end-to-end on one host, applies resource limits and documents a role inventory.

You'll build

A single host serving web, file and database roles with per-role resource limits.

Tools: nginx, samba, mariadb-server, curl

Server Roles: Web, File, and Database

STEP 1 OF 8

1

Install the three role packages

```
$ apt update && apt install -y nginx samba mariadb-server curl cifs-utils
```


Server Roles: Web, File, and Database

STEP 2 OF 8

2

Start nginx and serve a page (web role)

```
$ curl -s http://127.0.0.1/ | head
```

Server Roles: Web, File, and Database

STEP 3 OF 8

3

Add a name-based virtual host

```
$ ln -sf /etc/nginx/sites-available/site1 /etc/nginx/sites-enabled/
```

Server Roles: Web, File, and Database

STEP 4 OF 8

4

Create a Samba user and share (file role)

```
$ (echo 'P@ssw0rd!'; echo 'P@ssw0rd!') | smbpasswd -s -a alice
```

Server Roles: Web, File, and Database

STEP 5 OF 8

5

Connect to the Samba share

```
$ smbclient //127.0.0.1/share -U alice%P@ssw0rd! -c 'ls'
```

Server Roles: Web, File, and Database

STEP 6 OF 8

6

Create a MariaDB database and least-privilege user (database role)

```
$ mysql -e "CREATE DATABASE appdb;"
```

Server Roles: Web, File, and Database

STEP 7 OF 8

7

Apply systemd resource limits so roles don't starve each other

```
$ systemctl set-property mariadb.service MemoryMax=512M CPUQuota=80%
```

Server Roles: Web, File, and Database

STEP 8 OF 8



Document the role inventory to a file

Server Roles: Web, File, and Database

Test it

The learner verifies success by getting a page from nginx, listing the Samba share and running a SELECT against MariaDB.

Directory Services with OpenLDAP

LAB 11

The learner installs OpenLDAP, builds an OU/user/group tree, binds as a user and adds a group-based delegation ACL.

You'll build

An OpenLDAP directory with an OU structure, users, groups and a delegation ACL.

Tools: slapd, ldap-utils, ldapscripts

Directory Services with OpenLDAP

STEP 1 OF 9

1

Install and preseed slapd non-interactively

```
$ DEBIAN_FRONTEND=noninteractive apt install -y slapd ldap-utils ldapscripts
```

Directory Services with OpenLDAP

STEP 2 OF 9

2

Verify the directory is running

```
$ ldapsearch -x -H ldap://127.0.0.1 -b "dc=lab,dc=local" -s base "(objectclass=*)" namingContexts
```

Directory Services with OpenLDAP

STEP 3 OF 9

3

Create the people and groups OUs

```
$ ldapadd -x -D "cn=admin,dc=lab,dc=local" -w LabAdmin1! -f /tmp/ou.ldif
```

Directory Services with OpenLDAP

STEP 4 OF 9

4

Add user accounts

```
$ ldapadd -x -D "cn=admin,dc=lab,dc=local" -w LabAdmin1! -f /tmp/users.ldif
```

Directory Services with OpenLDAP

STEP 5 OF 9

5

Set a user password

```
$ ldapasswd -x -D "cn=admin,dc=lab,dc=local" -w LabAdmin1! -s 'AlicePass!' "uid=alice,ou=people,dc=lab,dc=local"
```

Directory Services with OpenLDAP

STEP 6 OF 9

6

Create groups and assign membership

```
$ ldapadd -x -D "cn=admin,dc=lab,dc=local" -w LabAdmin1! -f /tmp/groups.ldif
```

Directory Services with OpenLDAP

STEP 7 OF 9

7

Bind as a user and confirm identity

```
$ ldapwhoami -x -D "uid=alice,ou=people,dc=lab,dc=local" -w 'AlicePass!'
```


Directory Services with OpenLDAP

STEP 8 OF 9

8

Add a group-based delegation ACL

```
$ ldapmodify -Y EXTERNAL -H ldapi:/// -f /tmp/acl.ldif
```

Directory Services with OpenLDAP

STEP 9 OF 9

9

Map the concepts to Active Directory (OU=scope, group=role, ACL=delegation)

Directory Services with OpenLDAP

Test it

The learner verifies success by binding as alice and searching entries under ou=people.

Performance Monitoring, Baselining, Event Logs

LAB 12

The learner captures CPU/memory/disk/network baselines, inspects and rotates event logs, and wires threshold alerting with Prometheus node_exporter.

You'll build

A saved performance baseline file and a node_exporter metrics endpoint for alerting.

Tools: sysstat, htop, iotop, fio, prometheus-node-exporter

Performance Monitoring, Baselineing, Event Logs

STEP 1 OF 8

1

Install the monitoring tools and enable sysstat

```
$ apt update && apt install -y sysstat htop iotop fio prometheus-node-exporter
```

Performance Monitoring, Baselineing, Event Logs

STEP 2 OF 8

2

Read uptime and load averages

```
$ uptime
```

Performance Monitoring, Baselineing, Event Logs

STEP 3 OF 8

3

Capture a CPU baseline

```
$ mpstat 1 5
```

Performance Monitoring, Baselineing, Event Logs

STEP 4 OF 8

4

Capture a memory baseline

```
$ sar -r 1 5
```


Performance Monitoring, Baselineing, Event Logs

STEP 5 OF 8

5

Generate I/O load and measure IOPS and latency

```
$ iostat -xz 1 5
```

Performance Monitoring, Baselineing, Event Logs

STEP 6 OF 8



6

Save a combined baseline file for later diffing

Performance Monitoring, Baselineing, Event Logs

STEP 7 OF 8

7

Inspect event logs and configure rotation/retention

```
$ journalctl --since "1 hour ago" -p err
```

Performance Monitoring, Baselineing, Event Logs

STEP 8 OF 8

8

Expose metrics with node_exporter for thresholds/alerting

```
$ curl -s http://127.0.0.1:9100/metrics | grep -E 'node_cpu_seconds_total|node_memory_MemAvailable|  
node_filesystem_free' | head
```

Performance Monitoring, Baselining, Event Logs

Test it

The learner verifies success by producing a baseline file and confirming node_exporter serves the CPU, memory and filesystem metrics.

Data Migration and Transfer

LAB 13

The learner transfers data with rsync, SCP and SFTP, applies fast-copy techniques and reviews exfiltration controls including bandwidth caps.

You'll build

A mirrored dataset transferred with rsync/SCP/SFTP plus egress controls.

Tools: rsync, openssh-client, lftp, pv

Data Migration and Transfer

STEP 1 OF 9

1

Install the transfer tools and start ssh

```
$ apt update && apt install -y rsync openssh-client openssh-server lftp pv
```

Data Migration and Transfer

STEP 2 OF 9

2

Set up source and destination trees

```
$ for i in $(seq 1 50); do dd if=/dev/urandom of=/srv/src/file$i.bin bs=1K count=$((RANDOM%200+10)) 2>/dev/null;  
done
```


Data Migration and Transfer

STEP 3 OF 9

3

Mirror with rsync (archive, delta, delete)

```
$ rsync -avh /srv/src/ /srv/dst/
```

Data Migration and Transfer

STEP 4 OF 9

4

Rsync over SSH to a remote host

```
$ rsync -avzh -e ssh /srv/src/ user@remote:/srv/dst/
```

Data Migration and Transfer

STEP 5 OF 9

5

Copy with SCP (simple, no resume)

```
$ scp -r /srv/src/ root@127.0.0.1:/srv/dst-scp/
```

Data Migration and Transfer

STEP 6 OF 9

6

Transfer interactively over SFTP

```
$ sftp root@127.0.0.1
```

Data Migration and Transfer

STEP 7 OF 9



Compare cross-OS choices including Windows Robocopy

Data Migration and Transfer

STEP 8 OF 9

8

Fast-copy a large file showing throughput

```
$ pv /srv/src/file1.bin > /srv/dst/file1.bin
```

Data Migration and Transfer

STEP 9 OF 9

9

Apply a bandwidth cap to limit exfiltration

```
$ rsync -avh --bwlimit=1024 /srv/src/ user@remote:/srv/dst/
```

Data Migration and Transfer

Test it

The learner verifies success by confirming the destination mirrors the source and understands when to use SCP vs SFTP vs rsync.

High Availability: Clustering, Load Balancing, NIC Teaming

LAB 14

The learner builds an HAProxy round-robin load balancer over two backends, observes failover and failback, and configures keepalived VRRP and NIC bonding.

You'll build

An HAProxy load balancer with health checks plus keepalived and NIC-bond configs.

Tools: haproxy, keepalived, nginx, ifenslave

High Availability: Clustering, Load Balancing, NIC Teaming

STEP 1 OF 9

1

Install the HA packages

```
$ apt update && apt install -y haproxy keepalived nginx ifenslave apache2-utils
```

High Availability: Clustering, Load Balancing, NIC Teaming

STEP 2 OF 9

2

Start two backend web servers on different ports

```
$ nohup python3 -m http.server 8001 --directory /srv/back1 >/tmp/b1.log 2>&1 &
```

High Availability: Clustering, Load Balancing, NIC Teaming

STEP 3 OF 9

3

Configure HAProxy round-robin with health checks

```
$ systemctl restart haproxy
```

High Availability: Clustering, Load Balancing, NIC Teaming

STEP 4 OF 9

4

Test round-robin distribution across backends

```
$ for i in 1 2 3 4; do curl -s 127.0.0.1:8080; done
```

High Availability: Clustering, Load Balancing, NIC Teaming

STEP 5 OF 9

5

Switch the algorithm to least-conn

```
$ sed -i 's/balance roundrobin/balance leastconn/' /etc/haproxy/haproxy.cfg
```

High Availability: Clustering, Load Balancing, NIC Teaming

STEP 6 OF 9

6

Kill a backend to observe failover

```
$ kill $(lsof -ti :8001) || pkill -f '8001'
```

High Availability: Clustering, Load Balancing, NIC Teaming

STEP 7 OF 9



Configure keepalived VRRP heartbeat for a floating VIP

High Availability: Clustering, Load Balancing, NIC Teaming

STEP 8 OF 9

8

Configure NIC bonding (active-backup vs 802.3ad LACP)

```
$ modprobe bonding mode=active-backup miimon=100
```

High Availability: Clustering, Load Balancing, NIC Teaming

STEP 9 OF 9

A green circle with a white number 9 inside, indicating the current step in the process.

Review the cluster-aware patching procedure

High Availability: Clustering, Load Balancing, NIC Teaming

Test it

The learner verifies success by seeing traffic alternate across backends and only the surviving backend answer after a failover.

Virtualization with KVM / QEMU

LAB 15

The learner creates a thin-provisioned qcow2 disk, defines a VM with virt-install, inspects virtual networking and adds a second virtual switch.

You'll build

A defined KVM VM with a qcow2 disk and a second isolated virtual switch.

Tools: qemu-kvm, libvirt-daemon-system, virtinst, bridge-utils

Virtualization with KVM / QEMU

STEP 1 OF 9

1

Install the virtualization stack and start libvirtd

```
$ apt update && apt install -y qemu-system-x86 qemu-utils libvirt-daemon-system virtinst bridge-utils
```

Virtualization with KVM / QEMU

STEP 2 OF 9

2

Check whether the host CPU supports hardware virt

```
$ egrep -c '(vmx|svm)' /proc/cpuinfo
```

Virtualization with KVM / QEMU

STEP 3 OF 9

3

Create a thin-provisioned qcow2 virtual disk

```
$ qemu-img create -f qcow2 /var/lib/libvirt/images/vm1.qcow2 5G
```

Virtualization with KVM / QEMU

STEP 4 OF 9

4

Define a VM mapping each flag to objective 2.5

```
$ virsh list --all
```


Virtualization with KVM / QEMU

STEP 5 OF 9

5

Inspect the default virtual network and virbr0

```
$ virsh net-dumpxml default
```

Virtualization with KVM / QEMU

STEP 6 OF 9



6

Compare bridged vs NAT vs isolated networking

Virtualization with KVM / QEMU

STEP 7 OF 9

7

Add and start a second virtual switch

```
$ virsh net-start internal
```

Virtualization with KVM / QEMU

STEP 8 OF 9

8

Review CPU/memory/disk/NIC overprovisioning rules

```
$ nproc
```

Virtualization with KVM / QEMU

STEP 9 OF 9



Compare public, private and hybrid cloud models

Virtualization with KVM / QEMU

Test it

The learner verifies success by confirming the qcow2 disk is thin-provisioned and the second virtual network starts and appears in virsh net-list.

Scripting Basics for Server Administration

LAB 16

The learner writes Bash scripts covering variables, data types, conditionals, loops and real admin tasks like service management and bulk account creation.

You'll build

A set of Bash scripts for service checks, bulk account creation and host bootstrap.

Tools: bash, coreutils, systemctl

Scripting Basics for Server Administration

STEP 1 OF 8



1

Review the four script types and their platforms

Scripting Basics for Server Administration

STEP 2 OF 8

2

Write a script using variables, comments and environment exports

```
$ chmod +x 01-basics.sh && ./01-basics.sh
```

Scripting Basics for Server Administration

STEP 3 OF 8

3

Use integers, strings and arrays

```
$ chmod +x 02-types.sh && ./02-types.sh
```

Scripting Basics for Server Administration

STEP 4 OF 8

4

Write a disk-usage conditional with comparators

```
$ chmod +x 03-conditionals.sh && ./03-conditionals.sh
```

Scripting Basics for Server Administration

STEP 5 OF 8

5

Write for and while loops

```
$ chmod +x 04-loops.sh && ./04-loops.sh
```

Scripting Basics for Server Administration

STEP 6 OF 8

6

Write a service-management task script

```
$ chmod +x 05-services.sh && ./05-services.sh
```

Scripting Basics for Server Administration

STEP 7 OF 8

7

Write a bulk account-creation task script

```
$ chmod +x 06-accounts.sh && ./06-accounts.sh
```

Scripting Basics for Server Administration

STEP 8 OF 8



8

Write a bootstrap script with set -euo pipefail discipline

Scripting Basics for Server Administration

Test it

The learner verifies success by running each script and confirming the service, account and bootstrap tasks produce the expected output.

Asset Management, Documentation, and Licensing

LAB 17

The learner scripts asset discovery, captures an as-built config set, reviews BIA-driven metrics and the licensing model table, and validates a license count.

You'll build

A discovered asset CSV, an as-built config snapshot and a license-count report.

Tools: dmidecode, coreutils, spreadsheet

Asset Management, Documentation, and Licensing

STEP 1 OF 8



Auto-discover the asset details into a CSV

Asset Management, Documentation, and Licensing

STEP 2 OF 8



2

Review the asset life-cycle stages

Asset Management, Documentation, and Licensing

STEP 3 OF 8

3

Capture the running config as an as-built doc set

```
$ systemctl list-units --type=service --state=running > services.txt
```

Asset Management, Documentation, and Licensing

STEP 4 OF 8



4

Review BIA-driven metrics (MTBF, MTTR, RPO, RTO, SLA, uptime)

Asset Management, Documentation, and Licensing

STEP 5 OF 8



5

Draft a change-management entry template

Asset Management, Documentation, and Licensing

STEP 6 OF 8

6

Restrict permissions on sensitive documentation

```
$ chmod 750 /root/asbuilt
```

Asset Management, Documentation, and Licensing

STEP 7 OF 8



Review the Server+ licensing model cheat sheet

Asset Management, Documentation, and Licensing

STEP 8 OF 8



8

Run a license-count validation script for true-up

```
$ chmod +x /root/license-count.sh && /root/license-count.sh
```

Asset Management, Documentation, and Licensing

Test it

The learner verifies success by producing the asset CSV, the as-built config files and a socket/core/vCPU count for a true-up.

Recap — Server Administration

- You can now: 2.1 Install server operating systems
- You can now: 2.1 Partition/volume types (GPT vs MBR, LVM) and file systems
- You can now: 2.2 Configure servers to use network infrastructure services
- You can now: 2.2 Firewall, Ports
- You can now: 2.3 Server roles requirements (Web, File, Database)
- You can now: 2.3/3.3 Directory connectivity, scope-based delegation

TOPIC 03

Security and Disaster Recovery

Data & physical security · IAM · mitigation & hardening · decommissioning · backup & DR

Key Concepts — Security and Disaster Recovery

1

Data security

Encryption at rest (LUKS/BitLocker) and in transit (TLS/SSH); retention policies and the data lifecycle.

2

Physical security

Access controls, mantraps, badge/biometric systems, cameras and environmental controls for the server room.

3

Identity & access management

User accounts, groups, RBAC, least privilege, MFA and permission management.

4

Mitigation & hardening

Malware prevention, DLP, SIEM; OS updates, disabling unused services and host-based hardening.

5

Decommissioning

NIST 800-88 media sanitization — clear, purge, destroy; recycling and asset disposal records.

6

Backup & disaster recovery

Full/incremental/differential backups, the 3-2-1 rule, snapshots, replication and site failover; RPO and RTO.

Hands-On Labs — Security and Disaster Recovery

Labs 18–21

- Data at Rest (LUKS) and Data in Transit (TLS/SSH)
- Physical and Environmental Security Walk-Through
- Identity & Access Management for Server Administration
- Multi-Factor Authentication with TOTP (PAM)

Labs 22–25

- Auditing, Logging, and SIEM Basics
- OS and Application Hardening
- Patch and Update Management
- Secure Decommissioning and Media Destruction

Labs 26–27

- Backup Strategy: Full, Incremental, Differential, Snapshot
- Disaster Recovery: Replication and Site Failover

Data at Rest (LUKS) and Data in Transit (TLS/SSH)

LAB 18

The learner encrypts a volume with LUKS, proves the ciphertext is unreadable, serves TLS with a self-signed cert and hardens SSH.

You'll build

A LUKS-encrypted volume and a TLS-enabled nginx site with hardened SSH.

Tools: cryptsetup, openssl, openssh-server

Data at Rest (LUKS) and Data in Transit (TLS/SSH)

STEP 1 OF 9

1

Install the encryption tools

```
$ apt update && apt install -y cryptsetup openssl openssh-server
```


Data at Rest (LUKS) and Data in Transit (TLS/SSH)

STEP 2 OF 9

2

Format a volume at rest with LUKS

```
$ cryptsetup luksFormat /dev/loop20 --batch-mode --key-file=<(echo -n 'LabPass!1')
```

Data at Rest (LUKS) and Data in Transit (TLS/SSH)

STEP 3 OF 9

3

Open, format and mount the encrypted volume

```
$ cryptsetup luksOpen /dev/loop20 secret --key-file=<(echo -n 'LabPass!1')
```

Data at Rest (LUKS) and Data in Transit (TLS/SSH)

STEP 4 OF 9

4

Close it and prove the raw bytes are unreadable

```
$ strings /srv/secret.img | grep financials || echo "no plaintext visible — at-rest encryption works"
```

Data at Rest (LUKS) and Data in Transit (TLS/SSH)

STEP 5 OF 9

5

Inspect the cipher and LUKS header

```
$ cryptsetup luksDump /dev/loop20 | head -30
```

Data at Rest (LUKS) and Data in Transit (TLS/SSH)

STEP 6 OF 9

6

Generate a self-signed TLS cert and serve HTTPS

```
$ openssl req -x509 -newkey rsa:2048 -nodes -keyout /etc/ssl/private/lab.key -out /etc/ssl/certs/lab.crt -subj  
"/CN=lab.local" -days 365
```

Data at Rest (LUKS) and Data in Transit (TLS/SSH)

STEP 7 OF 9

7

Verify the negotiated TLS protocol and cipher

```
$ echo | openssl s_client -connect 127.0.0.1:443 -servername lab.local 2>/dev/null | grep -E "Protocol|Cipher\s+:"
```

Data at Rest (LUKS) and Data in Transit (TLS/SSH)

STEP 8 OF 9



8

Review SSH hardening lines and BIOS/GRUB passwords

Data at Rest (LUKS) and Data in Transit (TLS/SSH)

STEP 9 OF 9

9

Map a retention policy to data classification

Data at Rest (LUKS) and Data in Transit (TLS/SSH)

Test it

The learner verifies success by confirming no plaintext is recoverable from the closed LUKS image and TLS 1.2/1.3 is negotiated.

Physical and Environmental Security Walk-Through

LAB 19

The learner inspects environmental sensors the OS can see and walks the physical access-control layers, lock types, environmental controls and a security checklist.

You'll build

A completed physical-security audit checklist for a server room.

Tools: Im-sensors, smartmontools

Physical and Environmental Security Walk-Through

STEP 1 OF 9

1

Install the sensor tools

```
$ apt update && apt install -y lm-sensors smartmontools
```

Physical and Environmental Security Walk-Through

STEP 2 OF 9



2

Review the physical access-control layers (defence in depth)

Physical and Environmental Security Walk-Through

STEP 3 OF 9

3

Review lock and authenticator types (RFID, biometric)

Physical and Environmental Security Walk-Through

STEP 4 OF 9

4

Inspect environmental sensors the OS exposes

```
$ sensors 2>/dev/null | head -30
```

Physical and Environmental Security Walk-Through

STEP 5 OF 9

5

Read drive temperature via SMART

```
$ smartctl -a /dev/sda 2>/dev/null | grep -i temperature | head -3
```

Physical and Environmental Security Walk-Through

STEP 6 OF 9



6

Review cameras and guards as detective controls

Physical and Environmental Security Walk-Through

STEP 7 OF 9



Map the concentric data-centre security zones

Physical and Environmental Security Walk-Through

STEP 8 OF 9



8

Review the mantrap defence against tailgating

Physical and Environmental Security Walk-Through

STEP 9 OF 9



Complete the physical-security audit checklist

Physical and Environmental Security Walk-Through

Test it

The learner verifies success by reading available environmental sensor data and completing the physical-security checklist.

Identity & Access Management for Server Administration

LAB 20

The learner builds RBAC groups with segregation of duties, delegates via sudoers, enforces password policy and lockout, and audits access.

You'll build

Three RBAC roles with scoped sudoers rules, password policy and audit checks.

Tools: sudo, libpam-pwquality, chage, setfacl

Identity & Access Management for Server Administration

STEP 1 OF 9

1

Install the IAM tools

```
$ apt update && apt install -y sudo libpam-pwquality
```

Identity & Access Management for Server Administration

STEP 2 OF 9

2

Create three role groups and users (segregation of duties)

```
$ useradd -m -G sysops alice && echo 'alice:AlicePass!1' | chpasswd
```

Identity & Access Management for Server Administration

STEP 3 OF 9

3

Delegate scoped commands per role via sudoers

```
$ visudo -c
```


Identity & Access Management for Server Administration

STEP 4 OF 9

4

Test that each role can only run its own commands

```
$ sudo -u bob -i bash -c 'sudo systemctl restart mariadb && echo bob-ok'
```

Identity & Access Management for Server Administration

STEP 5 OF 9

A green circle with a white number 5 inside, indicating the current step in the process.

Enforce password length and complexity policy

Identity & Access Management for Server Administration

STEP 6 OF 9



6

Enforce account lockout with pam_faillock

Identity & Access Management for Server Administration

STEP 7 OF 9

7

Enforce password aging

```
$ chage -M 90 -m 1 -W 7 alice
```

Identity & Access Management for Server Administration

STEP 8 OF 9

8

Layer file ACLs on top of group bits

```
$ setfacl -m g:netops:r-x /srv/dbdata
```

Identity & Access Management for Server Administration

STEP 9 OF 9

9

Audit logins, group membership and account changes

```
$ getent group sysops dbops netops
```

Identity & Access Management for Server Administration

Test it

The learner verifies success by confirming a user is denied a command outside its role and password/lockout policy is enforced.

Multi-Factor Authentication with TOTP (PAM)

LAB 21

The learner adds a TOTP second factor to SSH with PAM, provisions a user's secret, generates codes from the CLI and tests the two-factor login.

You'll build

An SSH login enforcing a password plus a TOTP second factor.

Tools: libpam-google-authenticator, qrencode, oathtool

Multi-Factor Authentication with TOTP (PAM)

STEP 1 OF 9

1

Install the MFA tools and start ssh

```
$ apt update && apt install -y libpam-google-authenticator qrencode oathtool openssh-server
```

Multi-Factor Authentication with TOTP (PAM)

STEP 2 OF 9

2

Create a non-privileged user

```
$ useradd -m -s /bin/bash mfauser
```

Multi-Factor Authentication with TOTP (PAM)

STEP 3 OF 9

3

Provision the TOTP secret for that user

```
$ sudo -u mfauser google-authenticator -t -d -f -r 3 -R 30 -w 17 -Q UTF8
```

Multi-Factor Authentication with TOTP (PAM)

STEP 4 OF 9

4

Wire PAM and sshd to require TOTP

```
$ echo 'auth required pam_google_authenticator.so nullok' > /etc/pam.d/sshd.mfa
```

Multi-Factor Authentication with TOTP (PAM)

STEP 5 OF 9

5

Enable challenge-response and restart ssh

```
$ systemctl restart ssh
```

Multi-Factor Authentication with TOTP (PAM)

STEP 6 OF 9

6

Generate the current TOTP from the CLI for testing

```
$ oathtool --totp -b "$SECRET"
```

Multi-Factor Authentication with TOTP (PAM)

STEP 7 OF 9

7

Test the two-factor login

```
$ ssh -o PreferredAuthentications=keyboard-interactive,password -o PubkeyAuthentication=no mfauser@127.0.0.1
```

Multi-Factor Authentication with TOTP (PAM)

STEP 8 OF 9

8

Review scratch codes for account recovery

```
$ sudo tail -5 /home/mfauser/.google_authenticator
```


Multi-Factor Authentication with TOTP (PAM)

STEP 9 OF 9



Review the factor-category test for true MFA

Multi-Factor Authentication with TOTP (PAM)

Test it

The learner verifies success by confirming login is denied with a wrong TOTP code even when the password is correct.

Auditing, Logging, and SIEM Basics

LAB 22

The learner writes auditd rules for the four audit targets, configures rsyslog forwarding and rotation, runs logwatch and simulates a SIEM correlation rule.

You'll build

Auditd file-watch rules, log rotation policy and a simulated correlation alert.

Tools: auditd, rsyslog, logwatch

Auditing, Logging, and SIEM Basics

STEP 1 OF 8

1

Install and enable auditd and rsyslog

```
$ apt update && apt install -y auditd audispd-plugins rsyslog logwatch
```

Auditing, Logging, and SIEM Basics

STEP 2 OF 8

2

Write auditd rules for user/group/login/deletion events

```
$ augenrules --load
```

Auditing, Logging, and SIEM Basics

STEP 3 OF 8

3

Trigger and search an audit event

```
$ ausearch -k id_changes -ts recent | tail -20
```

Auditing, Logging, and SIEM Basics

STEP 4 OF 8

4

Review rsyslog severity levels and forwarding

```
$ logger -p auth.notice "Server+ lab test: notice-level entry"
```

Auditing, Logging, and SIEM Basics

STEP 5 OF 8

5

Configure log rotation and retention

```
$ logrotate -d /etc/logrotate.d/srvplus 2>&1 | tail
```


Auditing, Logging, and SIEM Basics

STEP 6 OF 8

6

Generate a daily report with logwatch

```
$ logwatch --output stdout --range today --detail Med | head -60
```

Auditing, Logging, and SIEM Basics

STEP 7 OF 8

7

Simulate a SIEM correlation rule with awk

```
$ awk '/sshd.*Failed/ {fails[$NF]++} /sshd.*Accepted/ {if (fails[$NF]>=5) print "ALERT: brute then success from", $NF}' /var/log/auth.log
```

Auditing, Logging, and SIEM Basics

STEP 8 OF 8

A green circle with a white number 8 inside, indicating the current step in the lab.

Review two-person integrity and separation of roles

Auditing, Logging, and SIEM Basics

Test it

The learner verifies success by finding the triggered auditd event and confirming the correlation rule and rotation config work.

OS and Application Hardening

LAB 23

The learner baselines with Lynis, minimises services/ports/packages, hardens nginx, deploys AIDE and AppArmor, and re-scans to watch the hardening index climb.

You'll build

A hardened host with a rising Lynis index, AIDE baseline and AppArmor enforcement.

Tools: lynis, apparmor-utils, aide, chkrootkit

OS and Application Hardening

STEP 1 OF 9

1

Install the hardening tools

```
$ apt update && apt install -y lynis apparmor-utils debsums aide chkrootkit unattended-upgrades
```

OS and Application Hardening

STEP 2 OF 9

2

Baseline the host with Lynis and note the hardening index

```
$ lynis audit system --quick 2>&1 | tail -60
```

OS and Application Hardening

STEP 3 OF 9

3

Disable unused services

```
$ systemctl disable --now "$s"
```


OS and Application Hardening

STEP 4 OF 9

4

Close unneeded ports with the firewall

```
$ ufw default deny incoming
```

OS and Application Hardening

STEP 5 OF 9

5

Remove unneeded packages (minimisation)

```
$ apt purge -y telnet ftp rsh-client 2>/dev/null
```

OS and Application Hardening

STEP 6 OF 9

6

Harden the nginx web role

```
$ nginx -t 2>/dev/null && systemctl reload nginx 2>/dev/null
```

OS and Application Hardening

STEP 7 OF 9

7

Build an AIDE file-integrity baseline and check

```
$ aideinit -y -f 2>&1 | tail
```

OS and Application Hardening

STEP 8 OF 9

8

Enforce an AppArmor profile (MAC)

```
$ aa-enforce /etc/apparmor.d/usr.sbin.nginx 2>/dev/null
```

OS and Application Hardening

STEP 9 OF 9

9

Re-scan with Lynis and compare the score

```
$ lynis audit system --quick 2>&1 | grep -E "Hardening index|warning|suggestion" | tail -20
```

OS and Application Hardening

Test it

The learner verifies success by confirming the Lynis hardening index improves after the minimisation and hardening steps.

Patch and Update Management

LAB 24

The learner distinguishes security from feature updates, snapshots and smoke-tests, applies patches with change discipline, and configures security-only auto-apply with a rollback plan.

You'll build

A security-only unattended-upgrades config plus a patch report and rollback pin.

Tools: unattended-upgrades, needrestart, debsecan

Patch and Update Management

STEP 1 OF 9

1

Install the patching tools

```
$ apt update && apt install -y unattended-upgrades needrestart apt-listchanges debsecan
```

Patch and Update Management

STEP 2 OF 9

2

Inventory current package versions as a baseline

```
$ apt list --upgradable 2>/dev/null | head -20
```

Patch and Update Management

STEP 3 OF 9

3

Identify security-only updates

```
$ apt-get -s dist-upgrade | grep -i security | head
```

Patch and Update Management

STEP 4 OF 9

4

Apply patches with change-management discipline

```
$ DEBIAN_FRONTEND=noninteractive apt -y -o Dpkg::Options::="--force-confdef" -o Dpkg::Options::="--force-confold"  
upgrade
```

Patch and Update Management

STEP 5 OF 9

5

Check which services need restarting after upgrade

```
$ needrestart -b
```

Patch and Update Management

STEP 6 OF 9

6

Check whether a reboot is required

```
$ [ -f /var/run/reboot-required ] && cat /var/run/reboot-required
```

Patch and Update Management

STEP 7 OF 9

7

Configure unattended-upgrades for security-only auto-apply

```
$ unattended-upgrade --dry-run -d 2>&1 | tail
```

Patch and Update Management

STEP 8 OF 9

8

Plan a rollback via package pinning

```
$ apt-mark hold nginx
```


Patch and Update Management

STEP 9 OF 9



Write a patch report for the change ticket

Patch and Update Management

Test it

The learner verifies success by confirming the security-only auto-apply config validates and a rollback pin holds the package version.

Secure Decommissioning and Media Destruction

LAB 25

The learner works the decommission checklist, wipes a loopback image with shred and nwipe, and reviews SSD erase, physical destruction and recycling.

You'll build

A securely wiped media image with a documented decommission and destruction trail.

Tools: shred, wipe, nwipe, dd

Secure Decommissioning and Media Destruction

STEP 1 OF 9

1

Install the wipe tools

```
$ apt update && apt install -y wipe nwipe secure-delete
```

Secure Decommissioning and Media Destruction

STEP 2 OF 9

A green circle with a white number 2 inside, indicating the second step of the process.

Review the decommission workflow checklist

Secure Decommissioning and Media Destruction

STEP 3 OF 9

3

Create a drive image with sensitive content

```
$ strings /srv/oldserver.img | grep -E "pii|cred" | head
```

Secure Decommissioning and Media Destruction

STEP 4 OF 9

4

Logically wipe with shred (single pass)

```
$ shred -v -n 1 -z /srv/oldserver.img
```

Secure Decommissioning and Media Destruction

STEP 5 OF 9

5

Confirm no plaintext is recoverable

```
$ strings /srv/oldserver.img | grep -E "pii|cred" | head || echo "no plaintext recoverable"
```


Secure Decommissioning and Media Destruction

STEP 6 OF 9

6

Multi-pass wipe with nwipe (DoD method)

```
$ nwipe --autonuke --nowait --method=dod /srv/oldserver.img 2>&1 | tail -10
```

Secure Decommissioning and Media Destruction

STEP 7 OF 9



7

Review SSD-specific ATA Secure Erase and crypto-erase

Secure Decommissioning and Media Destruction

STEP 8 OF 9



8

Review physical destruction methods (degauss, shred, crush, incinerate)

Secure Decommissioning and Media Destruction

STEP 9 OF 9

9

Review cable remediation and internal vs external recycling

Secure Decommissioning and Media Destruction

Test it

The learner verifies success by confirming the sensitive strings are unrecoverable after the wipe.

Backup Strategy: Full, Incremental, Differential, Snapshot

LAB 26

The learner runs full, incremental and differential backups with tar, a deduplicated synthetic full with restic, and validates a restore against the source.

You'll build

A validated backup set spanning full, incremental, differential and restic snapshots.

Tools: tar, rsync, restic, lvm2

Backup Strategy: Full, Incremental, Differential, Snapshot

STEP 1 OF 9

1

Install the backup tools

```
$ apt update && apt install -y rsync restic lvm2
```

Backup Strategy: Full, Incremental, Differential, Snapshot

STEP 2 OF 9

2

Prepare a live dataset and backup target

```
$ for i in $(seq 1 10); do dd if=/dev/urandom of=/srv/data/file$i.bin bs=1K count=$((RANDOM%50+5)) 2>/dev/null; done
```


Backup Strategy: Full, Incremental, Differential, Snapshot

STEP 3 OF 9

3

Take a full backup with tar

```
$ tar -czf /srv/backup/full-$(date +%F).tgz -C /srv data
```

Backup Strategy: Full, Incremental, Differential, Snapshot

STEP 4 OF 9

4

Take an incremental backup with a snapshot file

```
$ tar --listed-incremental=/srv/backup/snap.snar -czf /srv/backup/inc-$(date +%F).tgz -C /srv data
```

Backup Strategy: Full, Incremental, Differential, Snapshot

STEP 5 OF 9

5

Take a differential backup

```
$ tar --listed-incremental=/srv/backup/snap-full.snar -czf /srv/backup/diff-$(date +%F).tgz -C /srv data
```

Backup Strategy: Full, Incremental, Differential, Snapshot

STEP 6 OF 9

6

Take deduplicated synthetic-full snapshots with restic

```
$ restic -r /srv/backup/restic backup /srv/data
```

Backup Strategy: Full, Incremental, Differential, Snapshot

STEP 7 OF 9



Review media types and GFS rotation

Backup Strategy: Full, Incremental, Differential, Snapshot

STEP 8 OF 9

8

Restore side-by-side and to an alternate location

```
$ restic -r /srv/backup/restic restore latest --target /srv/restore-alt
```

Backup Strategy: Full, Incremental, Differential, Snapshot

STEP 9 OF 9

9

Validate integrity and diff the restore against source

```
$ restic -r /srv/backup/restic check
```

Backup Strategy: Full, Incremental, Differential, Snapshot

Test it

The learner verifies success by confirming the restic integrity check passes and the restored data matches the source.

Disaster Recovery: Replication and Site Failover

LAB 27

The learner replicates data async with rsync and near-sync with lsyncd, reviews sync/DRBD concepts, plans keepalived VIP failover and runs a tabletop DR exercise.

You'll build

An async and near-real-time replication pair plus a DR tabletop exercise plan.

Tools: rsync, lsyncd, keepalived, inotify-tools

Disaster Recovery: Replication and Site Failover

STEP 1 OF 9

1

Install the DR tools

```
$ apt update && apt install -y rsync lsyncd keepalived inotify-tools
```

Disaster Recovery: Replication and Site Failover

STEP 2 OF 9

A green circle with a white number 2 inside, indicating the second step of the process.

Review hot/warm/cold/cloud site types

Disaster Recovery: Replication and Site Failover

STEP 3 OF 9

3

Replicate asynchronously with rsync

```
$ rsync -avh --delete /srv/primary/ /srv/dr/
```

Disaster Recovery: Replication and Site Failover

STEP 4 OF 9

4

Configure near-real-time replication with Isyncd

```
$ systemctl restart isyncd
```

Disaster Recovery: Replication and Site Failover

STEP 5 OF 9

5

Confirm the change propagated to the DR copy

```
$ cat /srv/dr/app.txt
```

Disaster Recovery: Replication and Site Failover

STEP 6 OF 9



6

Review synchronous replication (DRBD, semi-sync DB) trade-offs

Disaster Recovery: Replication and Site Failover

STEP 7 OF 9



Plan a keepalived VIP failover for DR cut-over

Disaster Recovery: Replication and Site Failover

STEP 8 OF 9



8

Review the four DR test types

Disaster Recovery: Replication and Site Failover

STEP 9 OF 9

A green circle with a white number 9 inside, indicating the current step in the process.

Run the tabletop DR exercise and capture action items

Disaster Recovery: Replication and Site Failover

Test it

The learner verifies success by confirming a change on the primary appears in the DR copy within seconds via `lsyncd`.

Recap — Security and Disaster Recovery

- You can now: 3.1 Data security concepts (encryption at rest and in transit)
- You can now: 3.2 Physical security concepts
- You can now: 3.3 Identity and access management
- You can now: 3.3 Multifactor authentication
- You can now: 3.4 Data security risks and mitigation (log analysis, SIEM)
- You can now: 3.5 Apply server hardening methods

TOPIC 04

Troubleshooting

Structured methodology · hardware · storage · OS/software · network · DR validation

Key Concepts — Troubleshooting

1

Methodology

The CompTIA 6-step model: identify the problem, theorise, test, plan, implement & verify, document — change one thing at a time.

2

Hardware faults

Power issues, POST/beep codes, failed drives (smartctl), overheating and memory errors (memtester, sensors).

3

Storage faults

Degraded/failed arrays, full or corrupt file systems (fsck), I/O bottlenecks and open-file locks.

4

OS & software faults

Boot and service failures (systemd, journalctl), failed patches, dependency and permission errors.

5

Network faults

Latency, DNS and gateway misconfiguration, packet loss and cabling — diagnosed with ping, mtr, dig and tcpdump.

6

DR validation

Testing backups and restores, verifying failover, and confirming RPO/RTO are actually met.

Hands-On Labs — Troubleshooting

Labs 28–29

- Troubleshooting Methodology Walk-Through
- Troubleshooting Common Hardware Failures

Labs 30–31

- Storage Troubleshooting
- OS and Software Troubleshooting

Labs 32–33

- Network Connectivity Troubleshooting
- Security Troubleshooting

Troubleshooting Methodology Walk-Through

LAB 28

The learner breaks nginx, then applies the Server+ 7-step methodology end-to-end from identifying the problem through root-cause analysis and documentation.

You'll build

A resolved incident with an RCA and a documented post-incident record.

Tools: nginx, journalctl, ss, systemctl

Troubleshooting Methodology Walk-Through

STEP 1 OF 9

1

Manufacture a fault by breaking the nginx listen port

```
$ sed -i 's/listen 80;/listen 22;/' /etc/nginx/sites-enabled/default
```

Troubleshooting Methodology Walk-Through

STEP 2 OF 9

2

Identify the problem and scope by collecting logs

```
$ journalctl -u nginx -n 40 --no-pager
```

Troubleshooting Methodology Walk-Through

STEP 3 OF 9

3

Replicate the fault from the server itself

```
$ curl -sI http://127.0.0.1/ 2>&1 | head
```

Troubleshooting Methodology Walk-Through

STEP 4 OF 9

4

Back up the config before making changes

```
$ cp /etc/nginx/sites-enabled/default /root/default.pre-fix.$(date +%s)
```

Troubleshooting Methodology Walk-Through

STEP 5 OF 9

5

Establish and test a theory (port 22 already in use)

```
$ ss -tlnp 'sport = :22'
```

Troubleshooting Methodology Walk-Through

STEP 6 OF 9

6

Implement the solution, one change at a time

```
$ sed -i 's/listen 22;/listen 80;/' /etc/nginx/sites-enabled/default
```

Troubleshooting Methodology Walk-Through

STEP 7 OF 9

7

Verify full system functionality including dependencies

```
$ curl -sI http://127.0.0.1/ | head -1
```

Troubleshooting Methodology Walk-Through

STEP 8 OF 9



Add a preventive config-validation measure

Troubleshooting Methodology Walk-Through

STEP 9 OF 9



9

Perform a 5-whys root-cause analysis and document it

Troubleshooting Methodology Walk-Through

Test it

The learner verifies success by confirming nginx returns HTTP 200 and produces a documented RCA record.

Troubleshooting Common Hardware Failures

LAB 29

The learner reads event logs first, stress-tests CPU thermals, checks memory/ECC and SMART predictive failures, and maps POST/LED/olfactory cues to causes.

You'll build

A hardware fault decision tree exercised against event logs, sensors and SMART.

Tools: dmesg, smartmontools, lm-sensors, memtester

Troubleshooting Common Hardware Failures

STEP 1 OF 9

1

Install the hardware diagnostic tools

```
$ apt update && apt install -y dmidecode lshw smartmontools lm-sensors memtester stress-ng edac-utils
```

Troubleshooting Common Hardware Failures

STEP 2 OF 9

2

Check event logs first for hardware faults

```
$ journalctl -k | grep -iE 'mce|edac|temperature|thrott|over.?heat|ecc' | tail
```

Troubleshooting Common Hardware Failures

STEP 3 OF 9

3

Stress-test CPU and read thermals/utilisation

```
$ stress-ng --cpu $(nproc) --timeout 10s --metrics-brief 2>&1 | tail
```

Troubleshooting Common Hardware Failures

STEP 4 OF 9

4

Check memory errors, ECC and crash-dump terms

```
$ dmesg | grep -i mce | tail
```

Troubleshooting Common Hardware Failures

STEP 5 OF 9

5

Read SMART predictive-failure attributes

```
$ smartctl -a /dev/sda 2>/dev/null | head -40
```


Troubleshooting Common Hardware Failures

STEP 6 OF 9



6

Review PSU/fan faults via the BMC SEL

Troubleshooting Common Hardware Failures

STEP 7 OF 9

7

Review CMOS battery failure symptoms

```
$ chronyc tracking 2>/dev/null | head
```

Troubleshooting Common Hardware Failures

STEP 8 OF 9



8

Map POST codes and visual/auditory/olfactory cues to causes

Troubleshooting Common Hardware Failures

STEP 9 OF 9

9

Distinguish host-hardware faults from misallocated-VM symptoms

```
$ iostat -x 1 3
```

Troubleshooting Common Hardware Failures

Test it

The learner verifies success by mapping each Server+ 4.2 symptom to a concrete tool and reading SMART health for predictive failure.

Storage Troubleshooting

LAB 30

The learner diagnoses disk/inode-full and slow I/O, recovers a corrupt superblock, rebuilds a failed RAID drive, and reviews cache-battery and cable faults.

You'll build

Recovered filesystems and a rebuilt RAID array with a storage-fault tool map.

Tools: mdadm, e2fsprogs, lsof, iotop

Storage Troubleshooting

STEP 1 OF 9

1

Install the storage diagnostic tools

```
$ apt update && apt install -y mdadm lvm2 e2fsprogs xfsprogs parted gdisk smartmontools lsof iotop sysstat
```

Storage Troubleshooting

STEP 2 OF 9

2

Check capacity and inode utilisation

```
$ df -i
```


Storage Troubleshooting

STEP 3 OF 9

3

Diagnose slow I/O with iostat and iotop

```
$ iostat -xz 1 5
```

Storage Troubleshooting

STEP 4 OF 9

4

Corrupt and then recover a filesystem via a backup superblock

```
$ fsck.ext4 -b 32768 $LOOP || true
```

Storage Troubleshooting

STEP 5 OF 9

5

Create a RAID 5 array and fail a drive

```
$ mdadm /dev/md30 --fail /dev/loop32 --remove /dev/loop32
```

Storage Troubleshooting

STEP 6 OF 9

6

Add a spare and watch the array rebuild

```
$ mdadm /dev/md30 --add /dev/loop35
```

Storage Troubleshooting

STEP 7 OF 9



Review cache-battery failure and write slowdown

Storage Troubleshooting

STEP 8 OF 9

8

Read OS clues for cable/connector/backplane faults

```
$ dmesg | grep -iE "ata|scsi|sas|link rate|hard reset" | tail
```

Storage Troubleshooting

STEP 9 OF 9

9

Diagnose page/swap errors

```
$ swapon --show
```

Storage Troubleshooting

Test it

The learner verifies success by mounting the recovered filesystem and confirming the RAID array rebuilds in `/proc/mdstat`.

OS and Software Troubleshooting

LAB 31

The learner triages systemd services, fixes clock skew and broken dpkg state, reviews recovery boot modes, and uses Ansible to prevent config drift.

You'll build

A recovered service and package state plus an idempotent Ansible playbook.

Tools: systemd, strace, chrony, ansible

OS and Software Troubleshooting

STEP 1 OF 9

1

Install the OS troubleshooting tools

```
$ apt update && apt install -y strace chrony ansible
```

OS and Software Troubleshooting

STEP 2 OF 9

2

Triage a service: status, dependencies, failed units

```
$ systemctl list-units --type=service --state=failed
```

OS and Software Troubleshooting

STEP 3 OF 9

3

Diagnose a hanging service with strace

```
$ strace -p $PID -e trace=accept4,read,write -c -f -t -o /tmp/strace.log &
```

OS and Software Troubleshooting

STEP 4 OF 9

4

Diagnose and fix clock skew that breaks auth

```
$ chronyc -a 'burst 4/4'
```

OS and Software Troubleshooting

STEP 5 OF 9

5

Recover from a broken dpkg/patch state

```
$ dpkg --configure -a 2>&1 | head
```

OS and Software Troubleshooting

STEP 6 OF 9

6

Check missing dependencies and downstream failures

```
$ apt-cache rdepends openssl | head
```

OS and Software Troubleshooting

STEP 7 OF 9



Review recovery boot modes (rescue, emergency, snapshot)

OS and Software Troubleshooting

STEP 8 OF 9

8

Diagnose memory leak and set CPU affinity/priority

```
$ ps -eo pid,comm,rss,nice,psr --sort=-rss | head -10
```

OS and Software Troubleshooting

STEP 9 OF 9

9

Prevent config drift with an idempotent Ansible playbook

```
$ ansible-playbook playbook.yml 2>&1 | tail -10
```

OS and Software Troubleshooting

Test it

The learner verifies success by confirming the service and package state recover and the Ansible playbook reports no changes on a second run.

Network Connectivity Troubleshooting

LAB 32

The learner triages 'I can't reach the server' bottom-up through the OSI layers using ip, ping, traceroute, dig, nc and tcpdump.

You'll build

A repeatable bottom-up network triage exercised layer by layer.

Tools: iproute2, dnsutils, tcpdump, mtr-tiny

Network Connectivity Troubleshooting

STEP 1 OF 9

1

Install the network diagnostic tools

```
$ apt update && apt install -y iproute2 iputils-ping dnsutils tcpdump mtr-tiny traceroute nmap netcat-openbsd
```

Network Connectivity Troubleshooting

STEP 2 OF 9

2

Layer 1: check link, cable and NIC state

```
$ ethtool eth0 2>/dev/null | head
```

Network Connectivity Troubleshooting

STEP 3 OF 9

3

Layer 2: check NIC config and the ARP table

```
$ ip neigh
```

Network Connectivity Troubleshooting

STEP 4 OF 9

4

Layer 3: check addressing, gateway and routes

```
$ ip route get 8.8.8.8
```


Network Connectivity Troubleshooting

STEP 5 OF 9

5

Test reachability with ping, traceroute and mtr

```
$ mtr -rwc 5 8.8.8.8 2>/dev/null | head
```

Network Connectivity Troubleshooting

STEP 6 OF 9

6

Resolve names down the DNS chain

```
$ dig +short example.com
```

Network Connectivity Troubleshooting

STEP 7 OF 9

7

Test port reachability with nc and nmap

```
$ nc -vz example.com 80 2>&1 | head
```

Network Connectivity Troubleshooting

STEP 8 OF 9

8

Capture packets to find the silent direction

```
$ tcpdump -ni eth0 -c 10 'icmp or port 53' 2>&1 | head
```

Network Connectivity Troubleshooting

STEP 9 OF 9



Work the bottom-up decision tree

Network Connectivity Troubleshooting

Test it

The learner verifies success by walking a connectivity failure layer by layer and pinpointing the drop with tcpdump.

Security Troubleshooting

LAB 33

The learner audits open ports and process states, checks file integrity with AIDE, hunts privilege-escalation misconfigs and runs an anti-malware scan.

You'll build

A security triage covering ports, processes, file integrity and malware.

Tools: nmap, lsof, aide, chkrootkit, clamav

Security Troubleshooting

STEP 1 OF 9

1

Install the security troubleshooting tools

```
$ apt update && apt install -y nmap lsof aide chkrootkit clamav clamav-daemon auditd
```


Security Troubleshooting

STEP 2 OF 9

2

Audit open ports against the firewall policy

```
$ nmap -sT -p- -T4 127.0.0.1 2>/dev/null | head -30
```

Security Troubleshooting

STEP 3 OF 9

3

Classify process states: active, orphan, zombie

```
$ ps -eo pid,ppid,state,user,comm | awk '$3=="Z"
```

Security Troubleshooting

STEP 4 OF 9

4

Scan for rogue processes and rootkits

```
$ chkrootkit | head -30
```

Security Troubleshooting

STEP 5 OF 9

5

Baseline and check file integrity with AIDE

```
$ aide --check 2>&1 | head
```

Security Troubleshooting

STEP 6 OF 9

6

Hunt world-writable, SUID and writable-sudoers misconfigs

```
$ find / -xdev \( -perm -4000 -o -perm -2000 \) -type f 2>/dev/null | head
```

Security Troubleshooting

STEP 7 OF 9

7

Run an anti-malware scan with the EICAR test file

```
$ clamscan /tmp/scan/ 2>&1 | tail -10
```

Security Troubleshooting

STEP 8 OF 9

8

Triage 'cannot open file': perms, MAC policy, service health

```
$ getfacl /srv/share/
```

Security Troubleshooting

STEP 9 OF 9



Work the security decision tree

Security Troubleshooting

Test it

The learner verifies success by detecting the AIDE integrity drift and the EICAR test file with ClamAV.

Recap — Troubleshooting

- You can now: 4.1 Troubleshooting theory and methodology
- You can now: 4.2 Troubleshoot common hardware failures
- You can now: 4.3 Troubleshoot storage problems
- You can now: 4.4 Troubleshoot common OS and software problems
- You can now: 4.5 Troubleshoot network connectivity issues
- You can now: 4.6 Troubleshoot security problems



WRAP-UP

Course Summary & Next Steps

What You Achieved

1

Hardware & Storage

Racked and powered servers; built RAID 0/1/5/6/10 and LVM; deployed NFS/iSCSI; managed firmware and out-of-band.

2

Server Administration

Installed operating systems and file systems; configured IP/DNS/DHCP; ran web/file/DB/LDAP roles, HA, KVM and scripts.

3

Security & DR

Encrypted data at rest and in transit; managed IAM and MFA; hardened and audited hosts; wiped media; built backups and DR.

4

Troubleshooting

Diagnosed hardware, storage, OS, software and network faults with the CompTIA 6-step methodology.

5

Exam readiness

Mapped every lab to an SK0-005 exam objective across all four domains.

Preparing for the SK0-005 Exam

- Redo every lab from memory until the command workflow and flags are automatic.
- Review the 'What you learned' bullets in each lab and the Learner Guide.
- Practise the RAID capacity/fault-tolerance formulas, subnetting, and the 6-step troubleshooting model.
- Take the free CompTIA practice assessment for Server+ SK0-005.
- Book the exam via a Pearson VUE test centre or online proctoring.

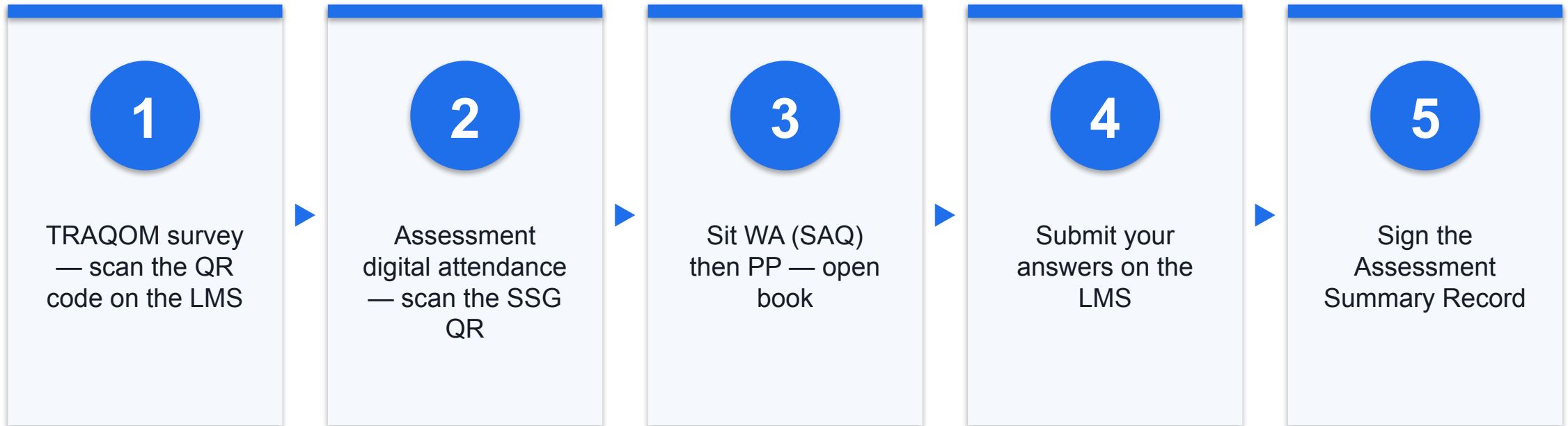
Practice Exam

- Sharpen your exam readiness with the Tertiary Infotech CompTIA Server+ practice exam.
- Practice exam: <https://exams.tertiaryinfotech.com/practice-exams/comptia/comptia-server-plus>
- Attempt it under timed conditions and review every explanation.
- Revisit any lab whose topic you miss, then re-take the practice exam.

Assessment

- Written Assessment (SAQ) — 1 hour. Practical Performance (PP) — 1 hour.
- Open book: slides, Learner Guide and approved materials only.
- Remember to take the Assessment digital attendance (TRAQOM).
- Submit your completed answers on the LMS at <https://lms-tms.tertiaryinfotech.com/>.

Assessment Flow



Digital Attendance (Mandatory)

- It is mandatory to take the AM, PM and Assessment digital attendance for WSQ-funded courses.
- The trainer/administrator displays the digital attendance QR code from the SSG portal.
- Scan the QR code with your mobile phone camera and submit your attendance.
- A minimum of 75% attendance is required to be eligible for assessment and funding.

YOU'RE SERVER+ READY

Thank You!

You are now ready to install, administer, secure and troubleshoot servers — and to sit the CompTIA Server+ (SK0-005) exam.